

Specifikace softwarového díla a časový plán implementace pro: **Simple code generation toolkit**

Jakub Turek

wip

Abstrakt

Projekt zaměřující se na implementaci jednoduché sady nástrojů pro generování kódu z pro projekt navrženého mezikódu (IR) v programovacím jazyce Haskell. Primárním cílem je prozkoumat různé techniky code generation a vytvořit nástroj, který je uživatelsky jednodušší než LLVM, ale zároveň poskytuje dostatečné možnosti, aby mohl být využit jako back-end pro překladač jazyka Haskell MicroHs, nebo výuku teorie překladačů.

Obsah

1	Základní informace	4
1.1	Popis a zaměření softwarového díla	4
1.2	Použité technologie	4
1.3	Konvence tohoto dokumentu	4
2	Stručný popis softwarového díla	4
2.1	Důvod vzniku softwarového díla a jeho základní části a cíle řešení	4
2.2	Hlavní funkce	4
2.3	Motivační příklad užití	5
2.4	Prostředí aplikace	5
2.5	Omezení díla	5
2.6	Dokumentace díla	5
3	Vnější rozhraní	5
3.1	Uživatelské rozhraní, vstupy a výstupy	5
3.2	Rozhraní s hardware	6
3.3	Rozhraní se software	6
4	Detailní popis funkcionality	6
4.1	Mezikód a jeho parsování	6
4.2	Code genration a optimalizace	7
4.3	Překladačový frontend	7
5	Ostatní (mimofunkční) požadavky	7
5.1	Požadavky na rozšiřitelnost a začlenitelnost	7
6	Ostatní požadavky	8
7	Negativní vymezení	8
8	Time-line & Milestones	8
A	Vymezení pojmů	9
B	To Be Determined List	9

Tabulka revizí

Jméno	Datum	Důvod změny	Verze
-------	-------	-------------	-------

1 Základní informace

1.1 Popis a zaměření softwarového díla

Software vzniklý v rámci projektu bude jednoduchý back-end překladače. Pro účely jehož implementace bude také definován vlastní mezikód (IR) vycházející z TAC (Three-address code). Back-end pak bude schopen pro vstup v podobě toho mezikódu vygenerovat ekvivalentní assembly pro architekturu X86. Z výsledného assembly bude následně pomocí assembleru a linkeru pro konkrétní platformu vygenerován spustitelný soubor. Za alternativu projektu je možné považovat LLVM, které nabízí právě vlastní IR a rozsáhlou na něj navázanou infrastrukturu pro tvorbu překladačů, primárně jejich back-endů. Předchozí práce s LLVM při tvorbě vlastního překladače, pak byla i hlavní motivací pro vznik celého projektu. Kdy jsem chtěl právě tuto externí závislost na robustním, ale komplexním nástroji jako je LLVM nahradit vlastní alternativou. Z čehož později po konzultaci s vedoucím projektu vzešla myšlenka vytvořit zjednodušenou alternativu právě k LLVM, která sloužila pro podobné projekty, pro než je LLVM příliš komplexním nástrojem, případně nedokážou využít všech jím nabízených možností.

1.2 Použité technologie

Pro implementaci projektu bude využit programovací jazyk Haskell. Pro parsování textové reprezentace IR ze vstupního souboru bude využita Haskell knihovna *Megaparsec* pro tvorbu monádických parseru. Dále bude projekt využívat externí assembler a linker, pro převod vygenerovaných instrukcí v assembly do podoby binárních objektů/spustitelných souborů pro konkrétní platformu/operační systém.

1.3 Konvence tohoto dokumentu

Názvy externích nástrojů sloužících jako externí závislosti a knihoven jsou zvýrazněny kurzívou.

2 Stručný popis softwarového díla

2.1 Důvod vzniku softwarového díla a jeho základní části a cíle řešení

Jak již bylo řečeno úvodem primárním impulzem ke vzniku projektu byla práce s LLVM v rámci jiného projektu. Následovaná snahou primárně ze studijních důvodů LLVM nahradit vlastní implementací. Tato snaha eventuálně vyústila v myšlenku naimplementovat jednoduchý nástroj, který by nabízel základní funkcionalitu LLVM a mohl by být použit jako jeho alternativa pro obdobné malé a výukové projekty, které většinou využívají jen zlomek z nabízených možností, ale zároveň jsou zatíženy vysokou komplexností celého systému. Podobně jako LLVM bude projekt založen na vlastní IR a nástroji, který na něm zakládě vygeneruje odpovídající assembly pro cílovou architekturu (v tuto chvíli x86). Projekt pak bude možné využít jednak pro tvorbu zcela nových překladačů doplněním o front-endu generující IR projektu pro jazyk, pro nějž chce uživatel vytvořit překladač. Případně bude možné ho integrovat do již existujících projektů úpravou front-endu daného překladače a jím generovaného mezikódu, ať už jde LLVM IR v případě projektů využívajících LLVM, nebo třeba kombinátory v jazyce C, jako v případě již zmíněného MicroHs.

2.2 Hlavní funkce

Software vyvynutý v rámci projektu by měl sloužit jako nástroj pro generování strojového kódu. Samotný kód bude generován na základě vstupu v podobě souboru s textovou reprezentací IR. V tuto chvíli bude nástroj schopen generovat pouze instrukce pro platformu x86, ale jeho design bude umožňovat pozdější odvození verzí pro další platformy. Kromě samotného generatoru kódu bude v rámci projektu vytvořen také jednoduchý frontend překladače, který bude generovat požadované IR. A dohromady tak budou tvořit překladač daného jazyka pro architekturu x86. Hlavním cílem je tedy vytvořit codegeneration nástroj,

který bude možné využít jako překladačový backend dalšími projekty, a zároveň demonstrovat toto využití dodáním jednoduchého překladačového frontendu.

2.3 Motivační příklad užití

Typickým užitím výsledného softwaru je tvorba jednoduchého překladače nějakého programovacího jazyka pro architekturu x86. Pro takovéto využití je třeba projekt doplnit o překladačový frontend daného jazyka generující projektem definovanou IR. Příklad tvorby kompletního překladače s vytvořeným nástrojem bude demonstrován i jako jeden z výstupů projektu.

2.4 Prostředí aplikace

Podporovaným prostředím pro využití nástroje budou zařízení s architekturou x86 a operačním systémem Linux, případně Windows. Jako externí závislosti bude pro své fungování program vyžadovat nástroje *NASM* a *ld* v případě Linuxového prostředí, případně *NASM* a *Link.exe* v případě Windows. Dále pak bude vyžadována Haskell knihovna *Megaparsec*.

2.5 Omezení díla

Popište veškerá omezení, která mohou ovlivnit specifikaci a implementaci. To mohou být například licenční podmínky využívaných technologií, omezení daná rozhraním hardwarové platformy nebo software, se kterým má dílo spolupracovat, vyžadovaná bezpečnost, konvence již existujících částí v případě rozšiřování funkcionality atd. Další kategorií je závislost na třetích stranách, například nutnost poskytnutí práv na specifický stroj, přístup do sítě, poskytnutí dat apod.

2.6 Dokumentace díla

Jelikož je projekt koncipován jako jednoduchý codegenration nástroj, tak bude třeba kromě běžné uživatelské a vývojové dokumentace dodat také specifikaci používané IR. Tato specifikace by zřejmě mohla být součástí běžné uživatelské dokumentace. Zároveň se ale jedná o rozsáhlejší standard definovaný v rámci projektu, jehož znalost je sice nutná pro plnohodnotné využití při vývoji vlastního překladače, nikolovi však pro samotné užité nástroje k vygenerování kódu z již existujících souborů s IR. Proč se jeví lepším specifikaci tohoto standardu vyjmout do samostatného dokumentu. Tento dokument by pak měl společně s uživatelskou dokumentací a vzorovým frontendem překladače dodaným vrácí projektu poskytovat dostatek informací nutných pro tvorbu vlastního překladače s využitím tohoto projektu.

3 Vnější rozhraní

3.1 Uživatelské rozhraní, vstupy a výstupy

Interakce s uživatelem bude probíhat prostřednictvím běžného CLI. Program bude nabízet přepínače pro funkcionality běžné u ostatních překladačů jako jsou

- Specifikace výstupního souboru
- Nastavení úrovně optimalizací
- Typ výstupu executable/assembly/object file
- Specifikace knihoven k linkování

Vstupem programu pak bude soubor obsahující textovou reprezentaci IR definované v rámci projektu. Na základě tohoto vstupu a příslušných přepínačů bude programem vygenerován výstupní soubor v podobě, spustitelného souboru programu definovaného vstupním IR, texového souboru obsahující vygenerované assembly, object file k nalinkování.

3.2 Rozhraní s hardware

Projekt nevyužívá žádný specifický hardware. Projekt se však zaměřuje na generování strojového kódu pro architekturu x86. Projekt tedy v tuto chvíli podporuje pouze zařízení s touto architekturou procesoru, respektive jeho výstupy v podobě spustitelných souborů, či assembly (využívající Intel syntaxi) jsou určeny pro tuto platformu.

3.3 Rozhraní se software

Projekt je koncipován jako jednoduchý backend překladače, z čehož plyne nutnost jeho integrace s dalšími kroky překladače kódu. Prvním z rozhraní je interakce s překladačovým frontendem a získání vstupního IR reprezentující překládaný zdrojový kód. Výměna dat na tomto rozhraní bude probíhat prostřednictvím textového souboru, který bude obsahovat jednotlivé IR instrukce zapsané v textové čitelné podobě. Frontend tedy zpracuje překládaný kód a na jeho základě vygeneruje odpovídající IR (instrukce a jejich serializace bude specifikována později), kterou následně zapíše do výstupního souboru. Poté spustí tento backend a předá mu cestu k tomuto výstupnímu souboru.

Druhým softwarovým rozhraní je rozhraní mezi backendem a assemblerem, který z vygenerovaných instrukcí/assembly vygeneruje object file ve formátu pro cílový operační systém. Výměna dat na tomto rozhraní bude opět probíhat prostřednictvím dočasného textového souboru, kdy backend opět uloží x86 assembly vygenerované pro vstupní IR do výstupního souboru a následně zavolá externí assembler, kterému předá tento soubor jako vstup.

Posledním softwarovým rozhraním projektu je rozhraní mezi backendem a linkerem, kde opět dojde k výměně dat prostřednictvím souborů. Backend jednoduše spustí externí linker s cestami k souborům, jenž je třeba nalinkovat (object file vygenerovaný assemblerem a případné knihovny specifikované uživatelem na vstupu), a linker následně zapíše výsledný nalinkovaný spustitelný soubor do backendem specifikovaného výstupního souboru. Toto rozhraní by projekt z koncepčního hlediska pravděpodobně ani nemusel obsahovat avšak výstupem backendu by mohl být nenalinkovaný object file. Avšak z uživatelského hlediska je pravděpodobně přívětivější doplnit backend o schopnost zavolat externí linker a vygenerovaný rovnou nalinkovat.

4 Detailní popis funkcionality

4.1 Mezikód a jeho parsování

V rámci projektu bude definován vlastní mezikód (IR), který bude sloužit jako vstup samotného nástroje. Cílem projektu je vyzkoušet různé alternativní způsoby reprezentace k reprezentaci využívané v rámci projektu LLVM. Definovaný mezikód bude pravděpodobně do jisté míry vycházet z Three-address code formátu.

Jednou z uvažovaných možností reprezentace je reprezentovat celý překládaný kód pomocí bloků instrukcí a control flow v kódu pak modelovat přímo jako operace nad instruction pointer registrem (tedy jako inkrementaci/dekrementaci instruction counteru o velikost příslušných bloků instrukcí). Takovýto přístup by měl negativní dopad na provádění řady optimalizací používaných právě LLVM a dalšími běžnými překladači, ale mohl by potenciálně zjednodušit IR a reprezentaci control flow v jejím rámci.

V případě, že by se žádný z alternativních přístupů neukázal jako dostatečně efektivní, navrátí by se projekt ke standardně využívané formě reprezentace pomocí mezikódu složeného z basic bloků v static single-assignment formě a reprezentaci control flow pomocí final instruction specifies the jump.

Aby bylo možné s kódem zapsaným pomocí takto definované reprezentace dále pracovat, je nutné v rámci projektu napsat také parser a serializer, který umožní efektivní převádění z a do textového zápisu. Parser by zároveň měl dokázat uživatele smysluplně upozornit na syntaktické chyby v textovém zápisu reprezentace, tak aby bylo snadné opravit případné chyby v jejím generování.

4.2 Code generation a optimalizace

Klíčovou funkcí je pak samotné generování x86 instrukcí na základě vstupního mezikódu. Nástroj v rámci tohoto kroku provede standardní úkony v podobě výběru instrukcí jejich plánování a alokace registrů. Výstupem tohoto kroku pak nebude přímo object file, ale jak již bylo řečeno x86 assembly využívající Intel syntaxi a pro jeho převod do podoby objektového souboru bude využito externích nástrojů.

Překladač by v rámci tohoto kroku měl provést taky základní optimalizace jako je constant folding and proopagation, dead code elimination, common subexpression elimination a podbně, které nejsou závislé přímo na cílové platformě. Tyto optimalizace by pak z důvodu usnadnění derivace variant pro jiné architektury než je x86 měli být implementovány na úrovni mezikódu ještě před samotným kódem generation tak, aby jejich implementace zůstala nezávislá na konkrétní architektuře.

4.3 Překladačový frontend

Posledním z větších problémů k vyřešení v rámci projektu je tvorba jednoduchého frontendu překladače, s jehož pomocí by bylo možné demonstrovat funkčnost projektu a jeho využití k tvorbě jednoduchého plnohodnotného překladače pro konkrétní jazyk.

V rámci této části bych chtěl integraci s frontendem demonstrovat na svém starším projektu jednoduchého frontendu pro vlatní toy language napsaném v jazyce OCaml, který v současné chvíli jako backend využívá právě LLVM. Modifikace tohoto projektu tak, aby namísto LLVM IR generoval IR definovanou v rámci projektu, by tedy mohla být dobrou ukázkou jak vyvíjený nástroj integrovat do již existujícího projektu, která by vzhledem k povaze projektu byla i dostatečně jednoduchá. Případně by bylo možné modifikovat již zmíněný překladač jazyka Haskell MicroHs, tak aby tento projekt využíval jako backend namísto dosavadního řešení s použitím kombinátorů v jazyce C.

5 Ostatní (mimofunkční) požadavky

5.1 Požadavky na rozšiřitelnost a začlenitelnost

Projekt je koncipován jako nástroj, který bude možné využít jako překladačový backend při tvorbě jednoduchého překladače (doplněním o frontend pro požadovaný jazyk). Z tohoto důvodu je tedy možnost snadné integrace projektu do již existujících/vznikajících projektů jednou z klíčových vlastností. Za tímto účelem projekt definuje vlastní IR standard, který takovouto integraci umožňuje. Pro integraci s projektem pak stačí, aby byl frontend, s nímž chce integraci provést, upraven tak, aby namísto doposud produkovaného výstupu generoval právě textovou reprezentaci IR definovanou tímto standardem, kterou pak již lze rovnou předat jako vstup tomuto projektu.

Ze stejných důvodů hraje důležitou roli i rozšiřitelnost projektu. Bylo by vhodné, aby bylo do budoucna možné projekt rozšířit o další cílové architektury jako například AArch64, nebo RISC-V. Takovýto požadavek na rozšiřitelnost je však mnohem komplikovanější na splnění, protože ze samotné podstaty problému musí být problém generování kódu (alokace registrů, výběr instrukcí) závislý na cílové architektuře. Nejlepším možným řešením pro dosažení co nejsnazší rozšiřitelnosti je tedy přesunout co možná nejvíce optimalizací do doby před samotným code generation a aplikovat je ještě na samotné IR (což ale není vždy možné). Rozšíření o novou podporovanou architekturu pak bude vyžadovat naimplementovat pouze code generation a pro architekturu specifické optimalizace bez nutnosti úpravy částí pracujících nad IR. Dalším problémem je v tomto případě závislost na externím assembleru pro generování objektových, který je také třeba nahradit alternativou pro danou architekturu (zde by bylo výhodnější využití *GNU assembleru*, který na rozdíl od *NASM* podporuje více architektur, ale zase není možné ho nativně využít na Windows).

6 Ostatní požadavky

Specifikujte ostatní požadavky na dílo.

7 Negativní vymezení

Zaměřením projektu je čistě generování kódu, tedy vygenerovat x86 assembly odpovídající vstupnímu programu zapsanému v IR. Projekt se tak nazabývá jednotlivými formáty objektových souborů ani jejich generování. Stejným způsobem se projekt nezabývá ani linkováním objektových souborů do podoby spustitelných souborů a formáty těchto souborů používaných jednotlivými operačními systémy. V řešení těchto problémů projekt spoléhá čistě na externí nástroje *NASM* a *ld*.

8 Time-line & Milestones

Specifikujte fáze definovatelné implementace — takzvané milestones, milníky. Uveďte datum, kdy bude dosaženo milníku a způsob, kterým bude prezentován vašemu vedoucímu, například prezentací, předvedením, commitem do repository apod. Typický rozestup mezi milníky by měl být přibližně jeden měsíc. Nezapomeňte na dostatečné rezervy. Není snadné přesně odhadnout, kolik budete potřebovat na vypracování jednotlivých částí času — programátoři často podceňují časové nároky třikrát i více!

Datum	Milník	Způsob prezentace
—	—	—

A Vymezení pojmů

IR - Zjednodušený mezikód, na nějž je překládaný kód převeden frontendem překladače, a slouží jako univerzální na jazyce nezávislá reprezentace, která funguje jako rozhraní mezi backendem a frontendem překladače.

B To Be Determined List

Uvedte seznam částí specifikace, které nebylo možno rozhodnout a popsat do doby dokončení tohoto dokumentu a budou dospecifikovány později. Při standardním průběhu projektu by tato část měla být nepotřebná.